

Telco Customer Churn Prediction

Beginner-Friendly Guide, Glossary & Interview Q&A

GitHub: github.com/SamirBeiruty/telco-churn-prediction

Read This First (in plain English)

Imagine you run a phone / internet company. Every month some customers cancel their service and leave. That is called **churn**. Losing a customer is expensive, because finding a new one costs far more than keeping an existing one.

This project is a computer program that looks at information about each customer (how long they have been with us, how much they pay, what plan they are on, etc.) and predicts **who is likely to cancel soon**. It is like a weather forecast, but instead of predicting rain, it predicts customer cancellations - so the company can reach out and try to keep those people before they leave.

To build it, the computer studies thousands of **past** customers where we already know who churned and who stayed. It learns the patterns ('customers on month-to-month plans leave a lot'), and then applies those patterns to new customers. That learning-from-examples process is called **machine learning**, and the whole field of using data this way is **data science**.

One-line summary to say out loud: "I used past customer data to teach a computer model to predict which customers will cancel, so the business can step in early and keep them."

Glossary - Every Term Explained Simply

The absolute basics

Data science — Using data (numbers, records, text) to answer questions and make predictions. A mix of statistics, programming, and domain knowledge.

Machine learning (ML) — Teaching a computer to find patterns from examples instead of programming every rule by hand. Show it thousands of past cases and it learns the pattern.

Model — The 'brain' that the computer builds after studying the examples. You feed it a customer and it outputs a prediction. Think of it as a very elaborate formula learned from data.

Algorithm — The recipe/method used to build the model (e.g., 'logistic regression'). The algorithm is the procedure; the model is the result.

Dataset — The table of data. Like a spreadsheet.

Row (record / sample) — One example. Here, one customer.

Column (feature / variable) — One piece of information about each example, e.g., monthly charge or contract type.

Feature — An input the model uses to make its prediction (a column we feed in). Example: tenure, monthly charges.

Target (label) — The thing we are trying to predict. Here it is 'Churn' (did the customer leave: yes or no).

Churn — When a customer cancels and stops being a customer. Our target.

Types of problems

Classification — Predicting a category/yes-no answer (e.g., churn vs stay). This project is classification.

Regression — Predicting a number instead of a category (e.g., predicting next month's revenue). Not what we do here, but good to know the contrast.

Binary classification — Classification with exactly two possible answers - here, churn (1) or stay (0).

Preparing the data

Data cleaning — Fixing problems in raw data: wrong formats, blanks, duplicates - so the model is not learning from garbage.

Missing values — Blank cells where data should be. Models cannot handle blanks, so we must fix them.

Imputation — Filling in missing values with a sensible substitute. We filled 11 blank 'total charges' with 0 because those were brand-new customers who had not been billed yet.

Numerical feature — A number, like tenure (months) or monthly charge.

Categorical feature — A label/category, like contract type ('Month-to-month', 'One year') or payment method.

Encoding — Converting text categories into numbers, because models only understand numbers.

One-hot encoding — A safe way to encode categories: make a separate yes/no (1/0) column for each category. Avoids tricking the model into thinking categories have an order (e.g., that 'Fiber' > 'DSL').

Scaling (standardization) — Putting numeric features on a comparable range (mean 0, similar spread) so a big-numbered column does not dominate just because its numbers are larger.

Feature engineering — Creating new, smarter columns from existing ones to help the model. Example: counting how many services a customer has.

EDA (Exploratory Data Analysis) — Looking at the data with charts and summaries to understand it and spot patterns before modeling.

Training and testing

Training set — The portion of data the model learns from (we used 80%).

Test set (hold-out) — Data the model never sees during training, used at the very end to check honestly how well it does (we used 20%). It is the 'final exam'.

Train/test split — Dividing the data into the training part and the test part.

Cross-validation (CV) — A more reliable way to score a model: split the training data into 5 chunks, train on 4 and test on 1, rotate 5 times, and average the scores. Reduces luck from a single split.

Fold — One of those chunks in cross-validation.

Stratification — Splitting the data so each part keeps the same churn ratio (~26.5%), so no split is accidentally too easy or too hard.

Hyperparameters — Settings you choose before training that control how the model learns (e.g., how many trees in a forest). Different from what the model learns on its own.

Overfitting — When a model memorizes the training data (including noise) and then performs badly on new data. Like a student who memorizes answers but cannot solve new problems.

Underfitting — The opposite: the model is too simple and misses real patterns even on the training data.

Data leakage — Accidentally letting information from the test set influence training, which makes scores look great but fail in the real world. The #1 mistake to avoid - we prevented it (see below).

The models we used

Logistic regression — A simple, fast model that weighs each feature and outputs a probability of churn. Easy to interpret. It won here.

Random forest — Many decision 'trees' (flowchart-like yes/no rules) that vote together. Captures complex patterns and is robust.

Gradient boosting / HistGradientBoosting — Builds many small trees one after another, each fixing the previous one's mistakes. Usually very strong on table data. (We used scikit-learn's version, similar to XGBoost.)

class_weight = 'balanced' — A setting that tells the model to care more about the rarer group (churners), so it does not ignore them just because they are the minority.

Measuring how good the model is

Probability — Instead of a flat yes/no, the model outputs a chance, e.g. '73% likely to churn'.

Threshold — The cutoff that turns a probability into a yes/no decision. Default is 0.5, but we tuned it to 0.60 to catch more churners.

Confusion matrix — A 2x2 table of correct vs incorrect predictions: true positives, false positives, true negatives, false negatives.

True positive (TP) — We predicted churn and the customer really did churn. A correct catch.

False positive (FP) — We predicted churn but the customer stayed. A false alarm (cheap mistake - we waste a retention offer).

False negative (FN) — We predicted stay but the customer churned. A miss (expensive mistake - we lost a customer we could have saved).

Accuracy — The % of all predictions that are correct. Misleading when classes are imbalanced (see below).

Class imbalance — When one outcome is much rarer than the other. Only 26.5% of customers churn, so guessing 'everyone stays' is 73% accurate but useless. That is why accuracy alone is a trap.

Precision — Of the customers we flagged as churners, how many really were? (Ours: 55%.) High precision = few false alarms.

Recall — Of the customers who actually churned, how many did we catch? (Ours: 72%.) High recall = few misses. We prioritized this.

F1 score — A single number balancing precision and recall (their harmonic mean). Useful when you care about both.

ROC curve — A graph showing the trade-off between catching churners and raising false alarms across all thresholds.

ROC-AUC — A single 0-1 score summarizing that graph. It is the chance the model ranks a random churner as riskier than a random non-churner. 0.5 = coin flip, 1.0 = perfect. Ours: 0.84.

Precision-Recall curve — Like the ROC curve but more informative when the positive class is rare.

Explaining the model

Feature importance — How much each input column matters to the predictions.

Permutation importance — Measure importance by shuffling one column and seeing how much the score drops. Big drop = important feature.

Coefficient — In logistic regression, a number per feature showing direction and strength: positive pushes toward churn, negative toward staying.

Project at a Glance

Problem	Predict whether a customer will churn (cancel) - yes/no
Dataset	IBM Telco Churn - 7,043 customers, 21 columns, 26.5% churn
Tools	Python, pandas, scikit-learn, matplotlib/seaborn
Models tried	Logistic Regression, Random Forest, HistGradientBoosting
How we checked	Stratified 80/20 hold-out + 5-fold cross-validation
Winner	Logistic Regression (simplest model that ties the rest)

Numbers to memorize (and what they mean)

Metric	Score	Plain-English meaning
CV ROC-AUC	0.846	Cross-validation score of the winning model
Test ROC-AUC	0.842	On unseen data, ranks a churner above a stayer 84% of the time
Recall (churn)	0.72	We catch ~72% of the customers who really churn
Precision (churn)	0.55	55% of the people we flag really do churn
F1 (churn)	0.63	Overall balance of precision and recall
Threshold	0.60	Our chosen cutoff (we did not just use 0.5)

Top churn drivers (the columns that mattered most): how long they have been a customer (tenure), their contract type, their internet service, and their monthly charge.

How the Project Works, Step by Step

1. Look at the data and find problems (data-quality audit)

Before trusting any data, we check its size, the type of each column, blanks, duplicates, and how common churn is. We found: a money column ('TotalCharges') was stored as text with 11 blanks, one column used 0/1 while others used Yes/No, and only 26.5% of customers churned.

2. Clean the data

- Converted the money column to real numbers and filled the 11 blanks with 0 (those were brand-new customers with no bill yet).
- Made the 0/1 column use 'No'/'Yes' like the others, for consistency.
- Turned the churn answer into 1 (churned) and 0 (stayed) so the model can use it.
- Removed the customer ID column - a random ID cannot help predict churn.

3. Explore the data (EDA)

We made charts of churn rate by group. Clear patterns appeared: people on month-to-month contracts, newer customers, fiber-optic internet users, and electronic-check payers churn the most.

4. Create smarter columns (feature engineering)

- num_services: how many add-on services each customer has (more services = more loyal).

- `tenure_group`: grouping 'how long they have stayed' into bands, because the effect is not a straight line.
- `avg_charges_per_month`: total spend divided by months, carefully avoiding division by zero for new customers.

5. Split into training and test sets

We kept 20% of customers aside as a 'final exam' the model never sees while learning. We split in a way that keeps the same churn ratio in both parts.

6. Prepare data + model together (a 'Pipeline')

We bundle the data preparation (scaling numbers, encoding categories) together with the model into one object. This is crucial: it makes sure the preparation is learned only from the training data, which prevents **data leakage**.

7. Compare three models fairly (cross-validation)

Using 5-fold cross-validation, we scored Logistic Regression, Random Forest, and Gradient Boosting. All scored about 0.83-0.85. Since the simple logistic regression tied the complex ones, we picked it.

8. Test it honestly + explain it

On the untouched test set we measured ROC-AUC, precision, and recall, tuned the decision cutoff to catch more churners, drew the confusion matrix and curves, and identified which features mattered most. Finally we turned the results into business advice.

The Big Ideas (explained with analogies)

Why accuracy can lie - and what we used instead

If only 26.5% of customers churn, a lazy model that always says 'nobody churns' is right 73% of the time - but it never catches a single churner, so it is worthless. That is why we judged the model on **recall** (did we catch the churners?) and **ROC-AUC**, not accuracy.

Data leakage - the classic trap

Imagine studying for an exam using the actual exam answers - you would score great, then fail a different test. Leakage is the data-science version: letting the model peek at test data while learning. We avoided it by preparing the data inside the Pipeline, so it only ever 'studies' the training portion.

Threshold tuning - choosing the cutoff

The model gives a probability (e.g., 73%). We must pick a cutoff to act on. The default 50% is rarely best. We chose 60% (using only training data) because it caught more churners - the outcome the business cares about.

Precision vs recall - the business trade-off

A **miss** (we think a customer will stay, but they leave) costs us a whole customer. A **false alarm** (we think they will leave, but they stay) only costs a cheap retention offer. So missing churners is the costlier error, and we tuned the model to reduce misses (higher recall).

The Code, Explained Chunk by Chunk

This is the actual code, broken into small pieces with a plain-English explanation of what each line does. If you can talk through these, you can explain the whole project. (A '#' in code is a comment - a note for humans that the computer ignores.)

1. Load the data and take a first look

```
import pandas as pd
df = pd.read_csv('data/raw/Telco-Customer-Churn.csv')
df.info()                                # column types + missing counts
df['Churn'].value_counts(normalize=True)  # how common is churn?
```

- **pd.read_csv(...)** reads the spreadsheet file into a table (called a 'DataFrame') and stores it in a variable named **df**.
- **df.info()** lists every column, its type, and how many values are filled - our first data-quality check. This is how we spotted the text/blank issues.
- **value_counts(normalize=True)** shows the share of Yes vs No - it told us 26.5% of customers churn (the imbalance).

2. Clean the data

```
def clean(df):
    df = df.copy()
    df['TotalCharges'] = pd.to_numeric(df['TotalCharges'],
                                       errors='coerce').fillna(0.0)
    df['SeniorCitizen'] = df['SeniorCitizen'].map({0: 'No', 1: 'Yes'})
    df['Churn'] = (df['Churn'] == 'Yes').astype(int)
    df = df.drop(columns=['customerID'])
    return df
```

- **def clean(df):** defines a reusable function so the exact same cleaning runs everywhere.
- **df.copy()** works on a copy so we never damage the original data.
- **pd.to_numeric(..., errors='coerce')** turns the text money column into real numbers; anything that is not a number (the 11 blanks) becomes empty (NaN). **.fillna(0.0)** then fills those blanks with 0 (new customers, no bill yet).
- **.map({0:'No', 1:'Yes'})** relabels the 0/1 column so it matches the other Yes/No columns.
- **(df['Churn'] == 'Yes').astype(int)** converts the target to 1 for 'Yes' and 0 for 'No', which the model needs.
- **.drop(columns=['customerID'])** removes the ID column - a random ID cannot help predict churn.

3. Create smarter columns (feature engineering)

```
df['num_services'] = (df[SERVICE_COLUMNS] == 'Yes').sum(axis=1)

df['tenure_group'] = pd.cut(df['tenure'],
                           bins=[-1, 12, 24, 48, 60, np.inf],
                           labels=['0-1yr', '1-2yr', '2-4yr', '4-5yr', '5yr+'])

df['avg_charges_per_month'] = np.where(df['tenure'] > 0,
                                       df['TotalCharges'] / df['tenure'],  # normal case
                                       df['MonthlyCharges'])               # if tenure is 0
```

- **(df[SERVICE_COLUMNS] == 'Yes')** makes a grid of True/False; **.sum(axis=1)** counts the Trues across each row = number of services the customer has.
- **pd.cut(...)** slices the continuous 'tenure' number into labeled bands (buckets), because churn risk is not a straight line in tenure.
- **np.where(condition, A, B)** means 'if condition then A, else B'. Here: divide total spend by tenure normally, but for brand-new customers (tenure 0) use the monthly charge instead, to avoid dividing by zero.

4. Split into training and test sets

```
from sklearn.model_selection import train_test_split
y = df['Churn']                # the target (what we predict)
X = df.drop(columns=['Churn']) # the features (what we predict from)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42)
```

- **y** is the answer we want; **X** is everything we use to predict it.
- **test_size=0.20** keeps 20% of customers aside as the untouched 'final exam'.
- **stratify=y** keeps the same 26.5% churn ratio in both the training and test parts, so the test is representative.
- **random_state=42** fixes the randomness so the split is identical every run (reproducible).

5. Bundle data-prep + model together (the leak-free Pipeline)

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline

preprocess = ColumnTransformer([
    ('num', StandardScaler(), numeric_cols),
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_cols),
])
pipe = Pipeline([('prep', preprocess), ('model', model)])
```

- **ColumnTransformer** applies different prep to different columns: scale the numbers, one-hot encode the categories.
- **StandardScaler** puts numeric columns on a comparable range so big numbers do not dominate.
- **OneHotEncoder(handle_unknown='ignore')** turns each category into its own 0/1 column and will not crash if it meets a new category later.
- **Pipeline** chains the prep and the model into one object. This is the most important line: during cross-validation the prep is re-learned on each training fold only, which is what prevents **data leakage**.

6. Compare models with cross-validation

```
from sklearn.model_selection import StratifiedKFold, cross_val_score
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(pipe, X_train, y_train,
                          cv=cv, scoring='roc_auc')
print(scores.mean(), scores.std())
```

- **StratifiedKFold(n_splits=5)** makes 5 folds (chunks), each keeping the churn ratio.
- **cross_val_score** trains and tests the pipeline 5 times (rotating which fold is the test) and returns 5 scores.
- **scoring='roc_auc'** judges by ROC-AUC, which is reliable on imbalanced data.

- **.mean()** is the headline score (0.846 for the winner); **.std()** shows how stable it is across folds.

7. Train the winner and get churn probabilities

```
pipe.fit(X_train, y_train)          # learn from training data
proba = pipe.predict_proba(X_test)[: , 1]  # probability of churn per customer
```

- **.fit(X_train, y_train)** trains the whole pipeline (prep + model) on the training data.
- **.predict_proba(X_test)[: , 1]** outputs each test customer's probability of churn. The **[: , 1]** grabs the 'Yes/churn' column (column 0 is 'stay').

8. Choose the decision cutoff (threshold tuning)

```
from sklearn.metrics import precision_recall_curve
prec, rec, thr = precision_recall_curve(y_train, train_proba)
f1 = 2*prec[:-1]*rec[:-1] / (prec[:-1]+rec[:-1]+1e-12)
best_thr = thr[f1.argmax()]
pred = (proba >= best_thr).astype(int)
```

- **precision_recall_curve** gives precision and recall at many possible cutoffs - computed on TRAINING data so we do not peek at the test set.
- **f1 = ...** calculates the F1 score at each cutoff. The tiny **+1e-12** just avoids dividing by zero.
- **f1.argmax()** finds the cutoff with the best F1; that becomes **best_thr** (about 0.60).
- **(proba >= best_thr).astype(int)** turns probabilities into final yes/no predictions using that cutoff.

9. Score the model honestly on the test set

```
from sklearn.metrics import roc_auc_score, classification_report
print('ROC-AUC:', roc_auc_score(y_test, proba))
print(classification_report(y_test, pred,
                           target_names=['Stay', 'Churn']))
```

- **roc_auc_score(y_test, proba)** compares predicted probabilities to the real answers - this produces the 0.84 score.
- **classification_report** prints precision, recall, and F1 for both classes in one table.

10. See which features matter most

```
from sklearn.inspection import permutation_importance
perm = permutation_importance(pipe, X_test, y_test,
                             scoring='roc_auc', n_repeats=10)
importances = pd.Series(perm.importances_mean,
                        index=X_test.columns).sort_values()
```

- **permutation_importance** shuffles each column 10 times and measures how much the ROC-AUC drops. A big drop means that column was important.
- **pd.Series(..., index=X_test.columns).sort_values()** labels each score with its column name and sorts them - revealing tenure, Contract, InternetService, and MonthlyCharges as the top drivers.

Interview Questions & Answers

Each answer is written the way you would say it. Terms used here are defined in the Glossary above.

Q. In simple terms, what is this project?

A. I built a tool that predicts which customers of a phone/internet company are about to cancel, using their account information. The company can then reach out and try to keep them. I used past customers - where we already know who left - to teach the model the warning signs.

Q. What is churn?

A. Churn is when a customer cancels and stops paying. Reducing churn is valuable because keeping a customer is much cheaper than finding a new one.

Q. What is a 'model' and how does it learn?

A. A model is a formula the computer builds by studying thousands of past examples. It finds patterns - like 'month-to-month customers churn a lot' - and uses them to score new customers. That learning-from-examples is machine learning.

Q. Walk me through the project end to end.

A. I audited the data for problems, cleaned them, explored the data with charts, created a few smarter columns, split the data into training and test sets, compared three models with cross-validation, tuned the decision cutoff for recall, tested honestly on unseen data, and explained which features mattered most. The winner was logistic regression at about 0.84 ROC-AUC.

Q. How did you handle the missing values?

A. Eleven rows had a blank total-charges value. They were all brand-new customers (zero months of tenure) who had not been billed yet, so the right value is 0. I filled them with 0 instead of deleting real customers, which would throw away information.

Q. What is data leakage and how did you avoid it?

A. Leakage is when the model accidentally sees test information while training, making it look better than it really is. I avoided it by bundling the data preparation and the model into one Pipeline, so the preparation is learned only from the training data during cross-validation.

Q. Why didn't you just use accuracy?

A. Because the data is imbalanced - only 26.5% churn. A model that always predicts 'stay' would be 73% accurate but catch zero churners. So I used recall and ROC-AUC, which actually measure how well we find churners.

Q. What is ROC-AUC, simply?

A. It is a single score from 0.5 (random guessing) to 1.0 (perfect) that says how well the model separates churners from non-churners. 0.84 means if you pick a random churner and a random stayer, the model rates the churner as riskier 84% of the time.

Q. What's the difference between precision and recall?

A. Recall: of everyone who actually churned, how many did we catch (72%). Precision: of everyone we flagged, how many really churned (55%). I focused on recall because missing a churner is more costly than a false alarm.

Q. Why did the simple model (logistic regression) win?

A. All three models scored about the same (~0.84), so I chose the simplest one. It is faster, easier to explain to non-technical people, and just as accurate. There was no benefit to a more complex model here.

Q. Why did you create new columns (feature engineering)?

A. To give the model better signals. For example, counting a customer's services captures loyalty in one number, and grouping tenure into bands reflects that churn risk drops sharply after the first year.

Q. What is cross-validation and why use it?

A. Instead of testing on one split (which could be lucky or unlucky), I split the training data five ways, trained and tested five times, and averaged the scores. It gives a more trustworthy estimate of performance.

Q. Why one-hot encoding for categories?

A. Models only understand numbers. One-hot makes a separate yes/no column for each category, so the model does not wrongly assume the categories have an order (like thinking 'Fiber' is greater than 'DSL').

Q. What is the threshold and why did you change it?

A. The model outputs a probability; the threshold is the cutoff for saying 'churn'. The default 0.5 is arbitrary, so I picked 0.60 using training data because it caught more churners, which is what the business wants.

Q. How do you know it isn't overfitting?

A. The cross-validation score (0.846) and the unseen test score (0.842) are almost identical. If it were overfitting, the test score would be much worse. I also used a held-out test set and regularized models.

Q. Which features matter most, and how do you know?

A. Tenure, contract type, internet service, and monthly charges. I measured this with permutation importance - shuffle a column and see how much the score drops; a big drop means the column was important.

Q. Why HistGradientBoosting instead of XGBoost?

A. It is scikit-learn's built-in version of the same idea (boosted trees). It is fast, needs no extra installation, and performed well - a clean, legitimate choice that keeps the project to one library.

Q. What business value does this provide?

A. It ranks customers by churn risk so the company can spend its retention budget on the people most likely to leave - especially month-to-month and new customers - instead of guessing.

Q. What would you improve next?

A. Add probability calibration for cost-based targeting, use SHAP to explain individual predictions, engineer interaction features, and add monitoring so we notice if the model's accuracy drifts over time once it is live.